The SMC interface: A Threat to Isolation of Virtualized Systems and Beyond!

Jean de Bonfils Lavernelle[1,2], Pierre-François Bonnefoi[1], Benoît Gonzalvo[2], and Damien Sauveron[1]

[1]XLIM, University of Limoges, 123 Av. Albert Thomas, Limoges, 87000, France
[2]Thales DIS, 147 Av. du Jujubier, La Ciotat, 13600, France
jean.de-bonfils-lavernelle@thalesgroup.com

***Abstract*—In ARM environments, the Secure Monitor firmware exposes an interface known as the Secure Monitor Call (SMC) interface to the OS kernel. Through this interface, the OS kernel can invoke services or request operations from the firmware for various purposes, such as cryptographic operations, power control, secure storage access, and managing device security features. In virtualized systems, it is crucial to restrict and protect this interface from VMs, given the number and criticality of the functions exposed through it. To that extent, SMC issued by guest OS kernels are intercepted by the hypervisor beforehand. This approach allows the hypervisor to enforce access control over the functions exposed to VMs by the firmware and maintain robust isolation. It ensures that the firmware does not process insecure monitor calls issued by untrusted VMs.**

This paper describes how the SMC interface is handled in virtualized systems, identifies associated risks, and explains why it is a significant attack vector for VMs. Specifically, it details why the support of the SMC interface in virtualized systems is a trade-off between security, features provided to VMs, and porting effort. We conducted an assessment of how the SMC interface is handled and supported in different open-source hypervisors for ARM-based platforms. This allowed us to provide an overview of this trade-off in the analyzed hypervisors across different ARM-based platforms. As a result of this process, we identified vulnerabilities at the hypervisor level that enabled VMs, through the SMC interface, to break the isolation ensured by the hypervisor. Finally, the criticality of this interface is further demonstrated by detailing security vulnerabilities (CVE-2025-0038) found during this assessment on Zynq UltraScale platforms. These security flaws enabled breaking TrustZone isolation and bypass the access control provided by hardware-based protection units.

***Index Terms*—Secure Monitor, Security, Hypervisor, Firmware, ARM**

I. Introduction

A. The Secure Monitor Call Interface

In ARM environments, the Secure Monitor firmware runs at EL3, the most privileged level, in the Secure world. It provides services to Non-Secure world software, such as an OS kernel or a hypervisor, through the Secure Monitor Call (SMC) interface. The OS kernel and hypervisor can request operations from the Secure Monitor through this interface for various purposes, including cryptographic operations, power control, secure storage access, and the management of device security features. To make Secure Monitor Calls and utilize the SMC interface exposed by the Secure Monitor, the ARM architecture defines the *smc* assembly instruction. When software running at lower privilege levels (i.e., EL1 or EL2) in the Non-Secure world executes a SMC, a synchronous exception is taken at EL3, transferring control to the Secure Monitor, which can then handle the request. In the ARM environment, the Secure Monitor running at EL3 in the Secure world can be Trusted Firmware-A (TF-A), an open-source reference implementation of Secure Monitor firmware for ARM Cortex-A platforms. This firmware typically manages the context switch between the Secure and Non-Secure worlds of ARM's TrustZone, which allows the separation of Secure and Non-Secure operations on the same hardware platform.

The ARM SMC Calling Convention (SMCCC) [14] is a specification that defines a standard interface for making SMCs in ARM-based systems. The SMCCC establishes a set of function calls that cover a range of operations, such as secure boot, power management, and cryptographic functions, ensuring that sensitive operations are handled securely. In TF-A, third parties can implement their services in the Secure Monitor, which can be exposed on specific ranges of the SMC interface. When issuing a SMCCC-compliant SMC, the software must store the Function ID (FID) in *w0* and the arguments for the requested service in the following registers (*r1-r5*). Each function that can be invoked through the SMC interface has a unique FID that identifies the Owning Entity Number (OEN). The OEN is a key part of the FID that designates the owner of a service or a set of functions accessible through the interface. This number enables distinguishing between various services accessible through the firmware and provided by different entities or components. For example, a Trusted OS or external hardware units on the System on a Chip (SoC) can provide services accessible through the firmware and the SMC interface. Both entities have an allocated range of FIDs identified by their respective OEN. Thanks to the OEN, the SMC is routed to the appropriate service handler implemented in the Secure Monitor. Each owning entity has a specific range of function identifiers that facilitates the management and handling of SMCs made to the firmware. The function number, also part of the FID, identifies which function within the service identified by the OEN is being requested. By utilizing the OEN, the SMCCC allows for modular and organized communication, thereby enhancing security and reducing the risk of conflicts between various services. Some ranges of functions identified by an

OEN are standard to the SMCCC interface, such as the Arm Architecture Service Calls (*OEN=0*) or the Standard Secure Service Calls (*OEN=4*), the latter of which allow access to standard sub-services like the Power State Coordination Interface (PSCI) [13] or the Software Delegated Exception Interface (SDEI) [11]. In contrast, some ranges identified by an OEN are specific to different parties. For instance, the Trusted OS Service interface exposes functions specific to the implemented Trusted OS, while the SiP Service Calls interface exposes functions specific to the SoC. Therefore, each of these different entities implements a service handler in the Secure Monitor in the function range identified by their OEN. Table II gives an overview of the different services exposed through the SMC interface. TF-A provides an EL3 runtime services framework that allows different services to be implemented interdependently within the Secure Monitor. The EL3 runtime services framework ensures that SMC functions are dispatched to their respective service implementations. This improves the scalability of the interface and simplifies the integration of various modular APIs, known as dispatchers. When a SMC is handled at the firmware level, it is forwarded to the relevant dispatcher in the Secure Monitor according to the OEN identified in the SMC. The dispatcher then handles the request and may forward it to another entity such as an external hardware unit to the CPU.

B. Context

In a virtualized environment with a hypervisor, not only can the hypervisor leverage services from the Secure Monitor through the SMC interface, but unprivileged isolated Virtual Machines (VMs) can do so as well. Given the number and criticality of functions exposed through this interface, it is clear that VMs should not be able to freely utilize this interface without hypervisor control. To that end, the ARM virtualization extension provides a means for the hypervisor to intercept the SMCs issued by guest VMs before they are handled by the Secure Monitor at the firmware level. By enabling the *HCR.TSC* register, any SMC issued at EL1 is trapped to EL2 and handled by the hypervisor. This mechanism ensures that a VM cannot issue an insecure SMC directly to the Secure Monitor. Consequently, the hypervisor can implement access control on the SMC interface, thereby ensuring that VMs cannot leverage functions exposed by the Secure Monitor that could allow them to break their isolation boundary. While this interface is a significant attack vector for VMs, it is also beneficial for them to access it, as it allows them to leverage features and essential functions exposed by various entities on the platform. Therefore, support for this interface at the hypervisor level is a trade-off between security, platform support, and features.

C. Organization

The remainder of the paper is structured as follows. Section II describes the different strategies implemented at the hypervisor level to handle the SMCs issued by the VMs. This section also explains why supporting this interface is a trade-off between security, platform support, and features. Additionally, it highlights why this interface is a particular attack vector for breaking isolation in virtualized systems. Then, Section III details key parts of the SMC interface and how they are handled in virtualized systems, with a strong focus on the corresponding attack surface exposed. Section IV exposes the extent to which the SMC interface can be leveraged by VMs in different representative open-source hypervisors for ARM-based platforms. This assessment allows for the evaluation of the attack surface of each hypervisor through the SMC interface across different platforms. To underscore the criticality of supporting this interface at the hypervisor level, the end of Section IV describes several security flaws found at the hypervisor level that compromised the isolation of VMs. Finally, Section V details security vulnerabilities at the firmware level (CVE-2025-0038 [26]) that we discovered on Zynq UltraScale+ platforms during this assessment. The security flaws enable Non-Secure software to break TrustZone isolation and a bus master to bypass the access control provided by hardware-based protection units. All these security flaws further support that this interface is a significant attack vector for breaking isolation at different levels of granularity. All security vulnerabilities found at the hypervisor and firmware levels were diligently reported and subsequently patched.

The salient contributions of this paper include:

- Provides an overview of key parts of the SMC interface in virtualized systems
- Explains how this interface is handled in virtualized systems and why it is a significant attack vector for VMs.
- Analyses the support of the SMC interface and the resulting attack surface in open-source hypervisors for ARM-based platforms.
- Highlights the importance of reviewing this interface at both hypervisor and firmware levels by detailing security flaws found during the analysis.

II. Management of Secure Monitor Calls in Virtualized Systems

A. Strategies Adopted

The SMC interface represents a significant attack vector for a VM to break the isolation ensured by the hypervisor and beyond. Thus, access to this interface by the VMs must be properly restricted by the hypervisor. On the other hand, the SMC interface exposes essential functions and features provided by the hardware or other software entities to the VMs. Ideally, the hypervisor should allow VMs to issue any SMC as long as the latter does not allow the VM to break its isolation boundary. However, this is challenging given the size of this interface and the diversity of functions exposed. To protect the SMC interface, a hypervisor typically implements a SMC handler. When a VM tries to issue a SMC through the SMC instruction, a SMC trap is triggered, which causes a context switch from the VM to the hypervisor, and the SMC handler is executed. Then, based on the requested function (FID), the OEN, the parameters of the SMC, and the requesting VM, the hypervisor can handle the SMC in three different ways.

a) Emulate: First, the hypervisor may emulate the SMC on behalf of the firmware. This can be required in certain cases, for example, the SMC allows the VMs to leverage essential features, but would represent a security risk if it was handled by the firmware. A hypervisor can also emulate a SMC to restrict certain features or the scope of a function exposed through the SMC interface to VMs. However, emulating functions exposed through the SMC interface requires significant porting efforts at the hypervisor level.

b) Forward: Next, if a SMC does not represent a security risk, it may be forwarded to the firmware. This avoids the porting effort at the hypervisor level required by emulation. Besides, the hypervisor cannot always emulate a monitor call, as some of them require capabilities on the platform that only the firmware possesses. For instance, the execution of some functions exposed through the SMC interface requires Secure access. Before forwarding the SMC, the hypervisor may implement mediation and access control based on the SMC, its parameters, and the VM.

c) Reject: Lastly, the hypervisor may also reject the SMC and directly return control to the VM. This occurs by default if the hypervisor does not support the SMC, as it cannot emulate the call and should not forward it to the firmware for security reasons. However, the VMs cannot benefit from the features provided through that SMC.

Figure 1 illustrates these three different strategies adopted by a hypervisor to handle SMCs from the VMs.

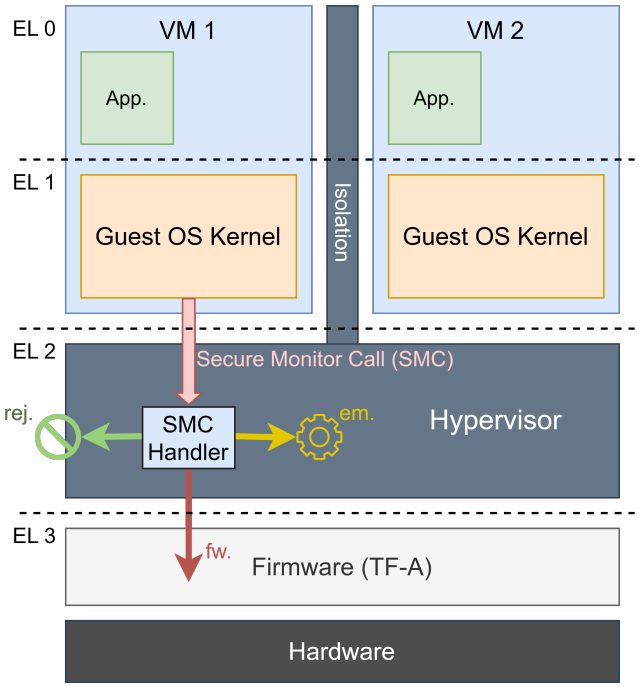

Fig. 1. Strategies for Handling SMCs at the Hypervisor Level. A SMC Can Be Either Rejected (Rej.), Emulated by the Hypervisor (Em.), or Forwarded to the Firmware (Fw.).

Table I summarizes the trade-offs of the three different strategies regarding security risk, impact on features provided to VMs, and support required at the hypervisor level. The SMC interface exposes a large attack surface on the firmware to VMs. For that reason, hypervisors implement a whitelist strategy by rejecting by default any SMC issued by untrusted VMs and forwarding only those that are necessary and considered secure. Certain hypervisors, like Jailhouse or Xen, rely on a privileged VM that can control the hypervisor and is responsible for managing essential hardware functions. As such, privileged VMs are trusted and need to access certain functions provided by the firmware; therefore, the hypervisor forwards more monitor calls issued by them than it would for regular VMs. As explained in Section III-B, it is convenient to rely on trusted VMs for this, as some services exposed by the SMC interface can be essential for the operation of the platform while being a particular attack vector. Therefore, the hypervisor may not necessarily adopt the same strategy depending on the VM that issued the SMC.

Strategy	Security risk	Impact on features	Support required
Rejected	Low	High	Low
Emulated	Medium	Low	High
Forwarded	High	Low	Medium

TABLE I
TRADE-OFFS OF THE THREE DIFFERENT STRATEGIES FOR HANDLING SMCS AT HYPERVISOR LEVEL

B. An Attack Vector for Breaking Isolation

Below are listed security considerations that highlight why the SMC interface is a significant attack vector in virtualized systems:

R1. The interface allows code execution by the Secure Monitor, which runs in the Secure world at the highest privilege level (EL3). The Secure Monitor has extended capabilities and access to the hardware. This could be exploited by a VM to indirectly manipulate critical system functions or gain unauthorized access to memory.

R2. The SMC interface allows the execution of functions by software that runs outside the hypervisor's control scope. Thus, it provides an opportunity for VMs to escape access control enforced by the hypervisor. In particular, SMCs may be forwarded to the firmware, which is responsible for isolation at a higher level of granularity than that of the hypervisor (e.g., TrustZone).

R3. The SMC interface allows code execution by the Secure Monitor, which is not aware of the different VMs' execution contexts. Therefore, VM isolation can no longer be guaranteed once the SMC is forwarded to the Secure Monitor. It is up to the hypervisor to ensure the consistency of SMCs from the different VMs to guarantee that VM isolation is maintained.

R4. The SMC interface allows access to functions provided by external units to the CPU which are not aware of the different execution and security states of the CPU (i.e., Exception Levels, Non-Secure access). These external hardware units treat all requests forwarded by the firmware in the same manner regardless of the initial caller.

R5. The SMC interface exposes a large attack surface and is constantly expanding. At the time of writing, the SMC interface comprises more than 128 different SMCs,

excluding specific interfaces (e.g., OEM, SiP, Trusted OS). The expansion of the SMC interface is supported by differences in the number of SMCs and interfaces defined through the various revisions of the ARM SMCCC.

R6. The SMC interface exposes services that are specific to various entities; therefore, supporting these services within the hypervisor requires advanced knowledge of the specific APIs associated with each entity. Moreover, since these services may vary from one environment to another, the hypervisor must implement specific handlers corresponding to the specific services exposed through the SMC interface.

All these security considerations explain why, upon receiving a SMC from a VM and before forwarding it to the firmware, the hypervisor must ensure that VMs' isolation will be maintained, by implementing mediation and access control. Figure 2 shows an overview of the SMC path with a virtualized system running in the Non-Secure world. This helps to visualize the criticality of the SMC interface in virtualized systems.

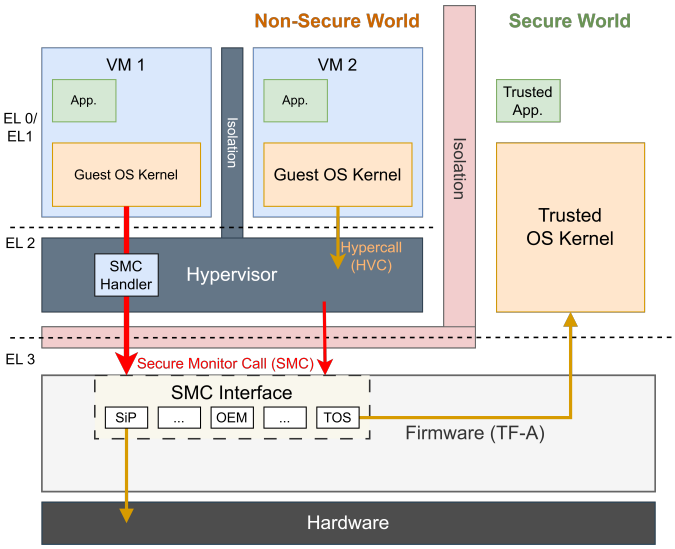

Fig. 2. Overview of SMC paths with a Virtualized Systems running in the Non-Secure world

III. Key Parts of the SMC Interface in Virtualized Systems

A. PSCI: An Essential Interface in Virtualized Systems

The Power State Coordination Interface (PSCI) is part of the Standard Secure Service Calls (*OEN=4*). In a native environment, it exposes essential functions to the OS kernel, providing power management controls on physical CPUs and the system. For instance, it allows booting secondary CPUs, suspending a CPU, or shutting down a CPU. In virtualized systems, VMs can leverage, to different extents, the same controls on the virtual CPUs (vCPUs) on which they run, instead of on the physical ones. In the case of a VM, system shutdown, reset, and suspend operations should only affect the VM and should not result in any physical power changes. Indeed, the hypervisor should maintain control over the system, and exposing these functions on the physical CPU to the VM would present clear security risks. Therefore, some PSCI calls are essential for the VM while being a straightforward attack vector if forwarded to the firmware. For that reason, the hypervisor cannot forward PSCI-related SMCs to the firmware without access control or mediation. However, given the essential functions exposed through this interface, the hypervisor must support it. For that reason, it has to emulate these functions, providing VMs with the same controls on their vCPUs that an OS kernel would have in a native environment on its physical CPUs. The necessity of the PSCI interface in virtualized systems is highlighted in Table II, which shows that the PSCI is the part of the SMC interface that is overall the most supported in the studied hypervisors.

B. SoC-specific Interfaces

The SiP Service Call interface exposes SoC-specific functions that provide various platform controls, such as clock and power management. This interface is often actively used by kernel drivers to manage hardware and leverage functions provided by external hardware units to the CPU. In complex SoCs with many peripherals, the SiP interface can be quite extensive. For example, on the Xilinx Zynq UltraScale+ MPSoC, the SiP Service Call interface exposes 45 SMCs. Moreover, it often exposes low-level functions that are not always well-documented. Restricting this interface at the hypervisor level is, therefore, challenging, as it requires SoC-specific knowledge.

As long as the SiP Service Call interface is supported by the hypervisor, it presents a large attack surface for VMs, potentially affecting the hypervisor, the firmware, and external hardware units to the CPUs (cf. Section V). Indeed, this interface allows access to functions provided by entities other than the firmware. In fact, some SMC SiP Service Calls are forwarded by the firmware to external hardware units. For instance, both the General Power Control (GPC) on NXP's i.MX8M SoCs and the Platform Management Unit (PMU) on Xilinx's Zynq UltraScale+ MPSoCs have functions exposed via the firmware through the SiP Service Call interface. Such hardware units, as well as the Secure Monitor, are not responsible for maintaining VMs isolation and are not aware of the virtualized system. In addition, these hardware units are not even aware of the different execution states of the CPU nor the different cores. Thus, it is the hypervisor's responsibility to ensure that VM isolation is maintained when forwarding a SMC to the firmware. This can be accomplished by implementing access control and mediation of SMCs at the hypervisor level before they are forwarded to the firmware. This includes parameter checks, necessary translations, verification of the context and resources of the VM that issued the SMC.

Additionally, the SiP Service Call interface impacts the portability of hypervisors to new SoCs. Indeed, this interface is specific to each SoC but may be necessary as it exposes essential controls and features of the platforms [17]. Thus, if the interface is restricted or not supported by the hypervisor,

it can significantly reduce the usability or function of the platform hardware by VMs or the overall platform operation. This further underscores the trade-off between security and features, as it presents a significant attack vector for VMs while being necessary for accessing useful platform controls and features. In a hypervisor that introduces a privileged VM (e.g., Xen, Jailhouse) considered trusted, this trade-off can be effectively managed. A straightforward solution can be implemented by forwarding any SiP Service Call from the privileged VM, which is considered trusted, while rejecting any SiP Service Call from other VMs. This approach ensures security and isolation, as only privileged VMs can access this interface. At the same time, privileged VMs can access the platform controls exposed through the SiP interface, ensuring correct platform operation and access to necessary platform features. This also significantly reduces the porting effort at the hypervisor level required for supporting this interface. Indeed, it does not require the hypervisor to implement access control and parameter validation on this interface, which would require advanced knowledge of the functionalities provided by each SoC through the SiP interface. In the case of a hypervisor that does not rely on a trusted VM, one VM could be allowed to access the SiP interface, but this would clearly impact the Trusted Computing Base (TCB) of the other VMs, as it would likely require other VMs to trust that VM.

C. Future Challenges and Security Considerations Related to Trusted Services

The ARM SMC interface is used as the principal channel for exposing services from the Secure world to the Non-Secure world. The firmware exposes the Trusted OS and Trusted Application Service interfaces, allowing Non-Secure software to leverage services from a Trusted OS running in the Secure world. The trusted OS OP-TEE has experimental virtualization support, which allows it to expose Secure services to different VMs while ensuring their isolation by maintaining different VMs' TEE states. For example, it can run different Trusted Applications from different VMs securely. However, a Trusted OS is not aware of the virtualized environment running in the Non-Secure world and cannot differentiate between the SMCs made by the hypervisor or different VMs. For this reason, the hypervisor has to perform upstream mediation to ensure that the Trusted OS can differentiate the requests from various VMs so that isolation can be maintained. To that extent, ARM introduced the client ID to the SMCCC, allowing a Trusted OS to distinguish between different guest OS instances as well as the hypervisor running in the Non-Secure world. The client ID of 0 is designated for SMCs from the hypervisor itself, while other values allow identifying the different VMs. The client ID is passed in a register (*w7*) when issuing a SMC. The client ID is expected to be created within the hypervisor, which uses it to register client guest OS instances to a Trusted OS. Additionally, the hypervisor must also perform all necessary address translation, as the Trusted OS is not aware of the translation between VMs' Intermediate Physical Address (IPA) and Physical Address (PA). All this work that must be implemented in the hypervisor is called a TEE mediator, and allows the Trusted OS to securely handle the requests from different VMs. This increases the porting effort at the hypervisor level, as the TEE mediator is specific to each Trusted OS that expose different functions to the Non-Secure world. This requires substantial mediation within the hypervisor. Firstly, the TEE mediator should be transparent to VMs, allowing them to use the API of the Trusted OS as if they were in a native environment. Furthermore, the hypervisor must ensure that the Trusted OS can securely manage requests from different VMs while preserving VM isolation in the Secure world. At the time of writing, only Xen implements a TEE mediator, meaning that only under this hypervisor should VMs be able to use the services of a Trusted OS. Without the TEE mediator, isolation would be compromised. A guest VM could leverage services with arbitrary client IDs, thus registering or unregistering on behalf of other VMs or the hypervisor. Moreover, without hypervisor mediation, services used by VMs and forwarded to the Trusted OS would likely result in faults or even complete system failure.

IV. Assessment of the Support of the SMC Interface in Virtualized Systems

A. Case of KVM: A Kernel-hosted Hypervisor

As a hosted hypervisor, KVM runs as part of the Linux kernel and provides two execution modes. The Virtualization-Host Extension (VHE) mode allows the entire Linux kernel to operate as a hypervisor at EL2. In Non-VHE mode, the Linux kernel runs at EL1, while a small part of the KVM hypervisor, referred to as the lowvisor, runs at EL2, leveraging the virtualization functions provided by the ARM architecture. It is worth mentioning that the SMCs issued by the kernel driver running in the host at EL1 do not take the same conduit as those issued by VMs. Monitor calls issued by the host Linux kernel are trapped by the lowvisor, and forwarded to the firmware, except some FFA and PSCI-related SMCs. Since KVM's trust model implies trusting the host Linux kernel, and because the SMC interface is necessary for system management and stability, the host Linux kernel is not restricted from issuing SMCs to the firmware.

In contrast, SMCs issued by VMs are handled at the kernel level by KVM. While some interfaces such as PSCI or True Random Number Generator (TRNG) are emulated, no SMC is forwarded to the firmware. KVM quite recently introduced support for enforcing SMC filtering [27]. The filter is defined as a set of non-overlapping ranges of FIDs, each specifying an action to be applied to SMCs within that range. SMCs issued by guest VMs can be **rejected**, **emulated** by the kernel, or **forwarded to user space**. This filtering mechanism enhances the security of the SMC interface by restricting SMCs to only those required by VMs. It also extends the support for the SMC interface for guest VMs by delegating the emulation of SMCs to user-space programs, such as QEMU. It also enhances flexibility, as it could enable user-space programs to define and modify the access control policy on the SMC interface on a per-VM basis during runtime.

TABLE II

SUPPORT FOR THE SMC INTERFACE IN XEN, BAO, AND JAILHOUSE FOR FAST CALLS. SMCS ISSUED BY VMS CAN BE EMULATED BY THE HYPERVISOR (EM.), FORWARDED TO THE FIRMWARE WITH PRIOR MEDIATION BY THE HYPERVISOR (M.), OR DIRECTLY FORWARDED TO THE FIRMWARE (FW.)

SMC services		Number of functions exposed	**Jailhouse**				**Xen**				**Bao**			
			Em.	M.	Fw.	Tot.	Em.	M.	Fw.	Tot.	Em.	M.	Fw.	Tot.
ARM Architecture [14]		8	2	1	0	3	3	0	0	3				
CPU Service		N/A												
SiP Service (i.MX8MQ)		8* (SoC-specific)	0	0	**All**	All								
SiP Service (ZCU)		45 (SoC-Specific)					3	31	**8**	42				
OEM Service		OEM-specific												
Standard Secure Service	PSCI [13]	22	7	0	0	7	9	0	0	9	5	1	0	6
	SDEI [11]	19												
	MM [6]	2												
	TRNG [10]	4												
	FFA [12]	33					5	4	0	9				
	ERRATA [8]	3												
	DRTM [15]**	9												
	PCI [7]**	5												
	CCA [24]	7												
Standard Hypervisor Service [9]		2		/	/			/	/			/	/	
Vendor Spec. Hypervisor Service		Hypervisor Specific		/	/			/	/		1	/	/	1
Vendor Spec. EL3 Monitor [25]		14 (TF-A)												
Trusted Application		Trusted OS Specific	0	0	**All**	All	1	9	0	10***				
Trusted OS														

*Each FID corresponds to a different handler, each of which exposes several interfaces that comprise around 23 functions in total

**These interfaces are not compliant according to the latest ARM SMCCC but they are specified as part of the specification in [7], [15]

***Values based on the Xen's TEE mediator for OP-TEE

B. Support of the SMC Interface in Xen, Bao, Jailhouse

1) Source for the Assessment: The support of the SMC interface in different open-source hypervisors for ARM-based platforms was assessed. This assessment was based on the latest GitHub tags as of October 2024. For Jailhouse, the assessment was based on the i.MX8MQ platform. Jailhouse (lf-6.1.55-2.2.2 tag) was evaluated from the official NXP i.MX-Jailhouse repository [22], which serves as the default source from which Jailhouse is built on these platforms (e.g., with Yocto). i.MX platforms were selected because Jailhouse serves as the reference hypervisor for these SoCs, and it is also on these platforms that Jailhouse has the best support for the SMC interface. For Xen, the assessment was conducted on a Xilinx Zynq UltraScale+ MPSoC platform (ZCU). The evaluation was based on the Xilinx Xen repository [29] (xlnx_rebase_4.18_2024.1 tag), as this is the default repository when building Xen with Yocto or PetaLinux. This repository is also used to build Xen provided in the pre-built version of the PetaLinux BSP. This is also the version of Xen that has the best support for the SMC interface, particularly the SiP Service Call interface. For Bao, the assessment was based on the official repository [21] (v1.0.0 tag). The data collected for Bao was not platform-specific, as it does not support any SoC-specific interfaces at the time of writing.

2) Methodology of Assessment: Table II presents an overview of the number of different functions exposed through the SMC interface for fast calls for each service. It provides details on the degree of support for this interface in three open-source hypervisors for ARMv8-A platforms: Jailhouse, Xen, and Bao. For each hypervisor, the total number of different functions available to unprivileged VMs is indicated (*Tot. in Table II*). The table also specifies the number of SMCs that are emulated at the hypervisor level (*Em. in Table II*), the number of SMCs forwarded to the firmware with prior mediation or access control by the hypervisor (*M. in Table II*) and the number of SMCs directly forwarded to the firmware (*Fw. in Table II*). Thus, SMCs forwarded to the firmware with prior mediation or access control by the hypervisor (*M. in Table II*) are differentiated from those directly forwarded to the firmware (*Fw. in Table II*). SMCs that are forwarded to the firmware with hypervisor mediation or access control (*M. in Table II*) are not considered emulated. Moreover, SMCs for which the hypervisor returns directly to the VMs because they are insecure or unsupported are also not considered emulated. However, SMCs for which the hypervisor solely returns a value for the requested service are considered emulated.

For each hypervisor, the columns "Em.," "M.," "Fw.," and "Tot." in Table II allow for the evaluation of the trade-offs made between security, features provided to VMs, and the porting effort required for specific interfaces. In Table II, the "Tot." column shows the extent to which unprivileged VMs can leverage services through the SMC interface. The greater the total number of functions supported by the hypervisor, the more VMs can benefit from features available through the SMC interface, whether they are emulated by the hypervisor or forwarded to the firmware. Overall, VMs in the studied hypervisors can leverage only a small part of the SMC interface. Table II confirms the necessity of the PSCI interface for virtualized systems. The "Em." and "M." columns show the porting effort that was required at the hypervisor level. Emulating a function at the hypervisor level enables features to be exposed to VMs without having to forward the SMC to the firmware, thereby mitigating this attack vector. This attack vector is also mitigated when the hypervisor implements access control and mediation for the SMCs before forwarding them to the firmware. In both cases, this requires porting effort at the hypervisor level but allows VMs to leverage features securely. The "Fw." column reveals the extent to

which unprivileged VMs can leverage services from the Secure Monitor without any access control or mediation from the hypervisor. Thus, this assumes that the related SMCs are always safe regardless of their parameters and the VM that issued them. For that reason, this is the main attack vector as supported by the following three subsections.

3) Summary of the Attack Surface: This table provides a clear view of the attack surface exposed to VMs for each service accessible through the SMC interface. In particular, Table II highlights two large attack surfaces for VMs on i.MX's Jailhouse: one on the i.MX8MQ's SiP Service Calls interface and another on the Trust OS Service Calls interface. Additionally, it underscores a significant attack surface on Xilinx's Xen through the ZCU SiP Service Calls interface. This assessment revealed two straightforward attack vectors that could compromise VMs' isolation in NXP's Jailhouse, as the next section explains. Regarding the ZCU's SiP Service Calls interface in Xen, we further investigated the functions forwarded to the firmware and exposed to VMs. We discovered one that could be exploited by a malicious VM to break isolation (cf. Section IV-E) as well as by Non-Secure software to break TrustZone (cf. Section V).

C. No Access Control on the Trusted OS Service Call Interface on i.MX Jailhouse

As explained in Section III-C, the Trusted OS Service Call interface should not be exposed to VMs without TEE mediators and, in particular, without any access control. In imx-jailhouse, it was discovered that the hypervisor forwarded any SMC issued to the Trusted OS Service Call interface to the firmware [19]. This raised a serious isolation flaw, as the hypervisor forwarded all such SMCs issued by the VM, regardless of whether it is a privileged VM (i.e., root cell) or another untrusted VM. Thus, a malicious VM was able to leverage services exposed by a Trusted OS without access control from the hypervisor. The functions exposed through the Trusted OS Service Call interface depend on the Trusted OS implemented in the Secure world, but a Trusted OS exposes functions that could allow a malicious VM to breach the isolation provided by the hypervisor (e.g., *OPTEE_SMC_FUNCID_BOOT_SECONDARY* in OP-TEE). Consequently, a malicious VM could exploit the functions provided by the Trusted OS, accessible through the Trusted OS Service Calls of the SMC interface, to break the isolation ensured by the hypervisor. The maintainers of imx-jailhouse were informed, and they subsequently patched the hypervisor to allow the Trusted OS Service Call interface to the root cell only [19]. In cases where the Trusted OS Service Call interface is exposed to untrusted guest VMs, the hypervisor should enforce access control specific to the Trusted OS implemented.

D. No Access Control on the SiP Interface on i.MX Jailhouse

In addition, as shown in Table II, there was no access control on the SiP interface in imx-jailhouse for i.MX8MQ platforms. This allowed any untrusted VM to issue SiP-related SMCs to the firmware without any restrictions from the hypervisor [18]. The SiP Service Call interface commonly exposes low-level functions that could easily be leveraged by malicious VMs to compromise VM isolation. For instance, any untrusted guest VM was able to leverage this interface on i.MX8MQ platforms to mask and unmask hardware IRQs, as well as modify IRQs affinity.

E. Broken Access Control of a SoC-specific SMC in Xen

1) Description of the Security Flaw on Xen: As Table II shows, Xen exposes a large attack surface on the firmware through the ZCU's SiP Service Call interface. Among the 45 functions exposed to unprivileged VMs, up to 39 can be forwarded to the firmware, including 8 without access control. This threat is significant because the functions exposed through the SiP interface allow VMs to manage hardware and low-level functions of the SoC. Additionally, the external hardware units that subsequently handle SiP-related functions are not virtualization-aware and run in TrustZone's Secure world.

Xen provides excellent support for this interface, enabling guest VMs to manage node devices, control clocks, and access special control registers under certain conditions. To this end, Xen implements fine-grained access control on the SiP Service Call interface to ensure that only safe SMCs are forwarded to the firmware. For instance, an unprivileged VM can disable a clock through the SiP Service Call interface only if it is attached to a device assigned to that VM. However, the VM cannot manage hardware that is not assigned to it. To that extent, Xen checks the FID of the SMC and its associated parameters, as well as the resources assigned to the VM.

The functions exposed to untrusted VMs through the SiP Service Call interface of the ZCU platform were carefully reviewed, with special focus on those directly forwarded to the firmware. This review aimed to ensure that these functions could not allow an unprivileged VM (i.e., any VM except Dom0) to breach its isolation boundaries through the firmware. A vulnerability was identified in which a SMC forwarded to the firmware by Xen allowed a VM to break the isolation ensured by the hypervisor. The security vulnerability was found in the Xilinx-Xen repository (*xlnx_rebase_4.18*) and affected Xen in the official PetaLinux BSP's pre-built version (v2024.1), as well as builds from Yocto and PetaLinux v2024.1. Xen maintainers were informed, and subsequently patched the flaw by restricting that SMC to Dom0 only [28].

Once the SMC, originally issued by a VM, has reached the firmware, VM isolation cannot be guaranteed. Indeed, the Secure Monitor is not aware of the execution contexts of the different VMs or the virtualization layer. Consequently, it is the hypervisor's responsibility to ensure the consistency of SMCs issued by the different VMs before forwarding the request to the Secure Monitor.

The firmware exposes some SMCs of the ZCU's SiP interface that allow an OS running at EL1 in the Non-Secure world to access FPGA functions and configure the Programmable Logic (PL) [1]. These SMCs are utilized by *fpga-util* in PetaLinux to load and store FPGA configurations. In Xen, while most of these SMCs are restricted to Dom0, the SMC *PM_FPGA_READ* was exposed to any VM and

forwarded by Xen to the firmware. This SMC allows an OS running at EL1 in the Non-Secure world to read the PL bitstream or configuration registers back to memory [1]. The function takes as arguments (i.e., in *x1* and *x2*) the memory address to which the PL configuration must be stored, the number of bytes to read, and a flag indicating whether PL configuration data or registers should be read back. If a VM issued *PM_FPGA_READ*, Xen forwarded the SMC to the firmware without first checking whether the memory space, pointed to by the pointer passed as an argument to the SMC, could be accessed by the VM. Consequently, a malicious VM was able to issue this SMC with an arbitrary memory pointer that points outside its memory, thereby overwriting that location with the PL data. This enabled a malicious VM to overwrite arbitrary memory locations, including other VMs' memory and the hypervisor's memory. However, it is worth mentioning that Xen prevented a VM from issuing the SMC *PM_FPGA_LOAD*, which allows loading a FPGA bitstream onto the PL. Therefore, a malicious VM could not control what data overwrites the designated memory location when leveraging the SMC *PM_FPGA_READ*. If no FPGA bitstream was loaded, this only cleared the designated memory region.

2) Exploitation of the Flaw for Breaking VM's Isolation: This flaw could be exploited to clear other VMs' memory or the hypervisor's memory, thereby causing a Denial of Service (DoS). However, it could be exploited by a malicious VM to further break isolation by disabling security controls on the platform and granting the VM read/write access to protected memory regions.

This flaw can be leveraged to clear the platform control register *IOU_INTERCONNECT _ROUTE*, which requires privileges [2] and is not accessible by any VM. This register enables routing all AXI (Advanced eXtensible Interface) transactions of certain DMA-capable peripherals, such as the Quad SPI or the SD host controller, to the SMMU [2]. In a virtualized system, the SMMU is configured by the hypervisor and is essential for enforcing access control for DMA-capable devices on memory. As explained in [16], it is crucial to prevent a DMA-capable device controlled by an untrusted VM from reading from or writing to memory locations that the VM itself cannot access.

Therefore, this control register is key for maintaining robust isolation on Zynq UltraScale+ MPSoCs, especially in a virtual environment. It ensures that all traffic from DMA-capable devices controlled by potentially malicious guest VMs cannot bypass the SMMU. If disabled, certain memory accesses generated by these DMA-capable devices would not pass through the SMMU. Consequently, a malicious VM controlling such a DMA-capable device could instruct it to read from or write to memory locations that the VM itself cannot access.

This privileged register cannot be directly accessed by the VM or any DMA-capable device assigned to a VM. First, because the hypervisor does not map the register into VMs' memory, and also because VMs running at EL1 and EL0 do not have sufficient privileges to access it. However, a VM was able to leverage the SMC *PM_FPGA_READ* to clear this register. This is because the external unit that ultimately performs the function has sufficient privileges to access this register (cf. Section V) and is beyond the scope of the access control enforced by the hypervisor.

Thus, a VM was able to issue the SMC *PM_FPGA_READ* with a memory pointer pointing to *IOU_INTERCONNECT_ROUTE* as a SMC argument, thus clearing the target register. Subsequently, some DMA traffic from DMA-capable devices that could be owned by VMs no longer passes through the SMMU, thereby bypassing the isolation provided by the hypervisor. It was then verified that a guest VM can effectively read from or write to memory locations outside its own memory through a DMA attack with a DMA-capable device that it controls. This exploit enabled access to other memory-mapped devices that could be owned by other VMs or the hypervisor, as well as gaining read/write access to the On-Chip Memory (OCM) and other control registers. This flaw could also be further exploited to disable security controls at the hypervisor level, for example, by clearing certain SMMU registers.

V. The SMC Interface: A Threat to TrustZone Isolation and Beyond

A. Details of the SMC path and Related Vulnerabilities

After further investigation, two security vulnerabilities were discovered which enables breaking isolation at higher levels of granularity through the SMC *PM_FPGA_READ*. First, it allowed Non-Secure software to access Secure memory and breach TrustZone isolation. Additionally, it enabled an untrusted processor to circumvent isolation and access control enforced by protection units known as the Xilinx Peripheral Protection Unit (XPPU) and Xilinx Memory Protection Unit (XMPU). These security vulnerabilities, for which a Coordinated Vulnerability Disclosure (CVD) process was initiated, impacted Kria SOM, Zynq UltraScale+ MPSoC, and Zynq UltraScale+ RFSoC platforms. The affected components were the PMU Firmware and Trusted Firmware-A (TF-A), generated through the AMD Software Development Kit (SDK), PetaLinux, or Vitis Software Platform versions 2017.3 to 2025.1. At the time of publication of this work, both vulnerabilities at the PMU and TF-A levels (i.e., CVE-2025-0038 and CVE-2025-48507) had been publicly released [3], [26]. CVE-2025-48507 has been assigned a CVSS severity score of **8.6 (High)**, while CVE-2025-0038 has a score of 6.6 (Medium). A Proof-of-Concept is publicly available [23] to demonstrate these vulnerabilities and show how they can be exploited by Non-Secure software to gain full access to TrustZone's Secure world.

Arm TrustZone is a hardware-based security architecture integrated into Arm processors, designed to create a Trusted Execution Environment (TEE). It partitions the system into two worlds: the Secure world and the Normal world. The Secure world runs sensitive tasks and protects critical assets such as cryptographic keys and personal data, while the Normal world handles general applications. This separation enhances security by ensuring that even if the Normal world is compromised, the Secure world remains protected. In TrustZone, a Secure master can access either Secure or Non-Secure slaves, while a Non-Secure master can only access Non-Secure slaves. A slave

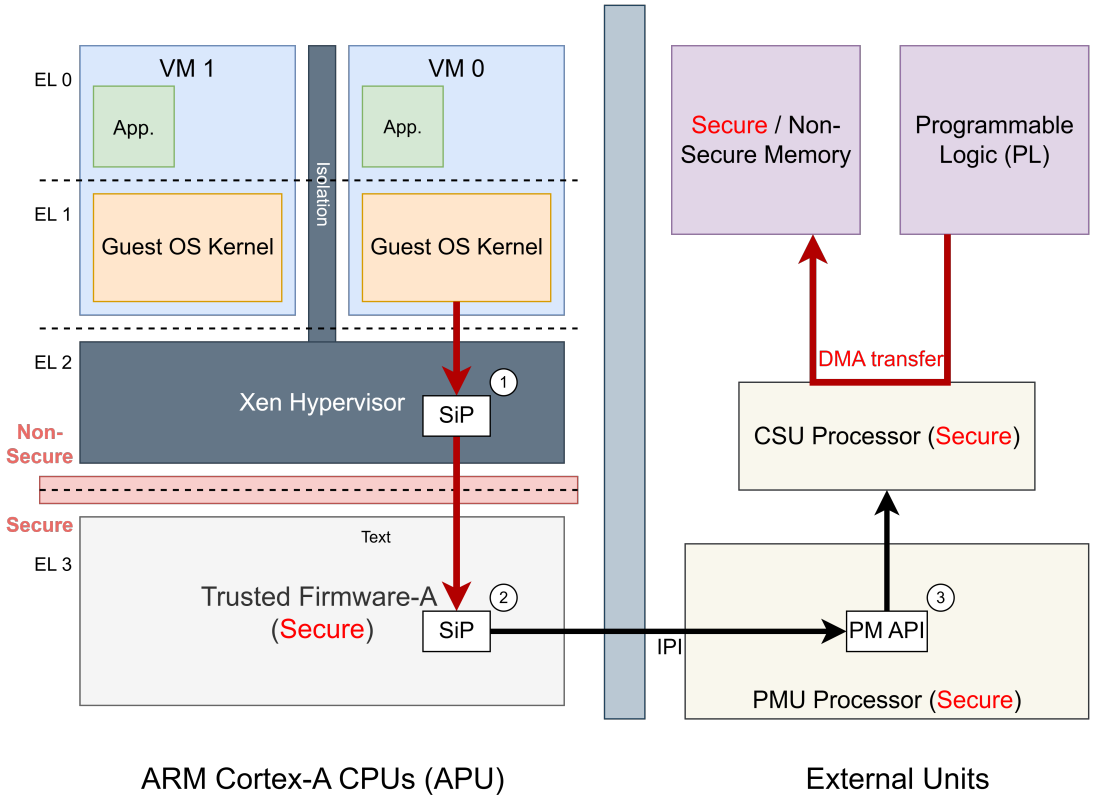

Fig. 3. Overview of the Full Path of the *PM_FPGA_READ* Request From a VM to the CSU DMA

is any resource that can be accessed by a master. It can be control registers of a device, a DMA channel, or memory (e.g., DDR DRAM, OCM). A master is any device or processor unit (e.g., DMA controller, CPU, DMA-capable I/O devices) that can access a slave. DMA-capable devices are both slaves and masters simultaneously; they can be controlled by software through the control registers on their slave interface, and they can access memory autonomously through their master interface. Some peripherals on the platforms are either Secure or Non-Secure, but most of them are usually Non-Secure by default and can be controlled by Non-Secure software. In addition, the security state of many peripherals can be switched through control registers (cf. "Table 16-2: TrustZone Profile" in [4]). Such registers require Secure access, and thus cannot be directly modified by Non-Secure software.

When Non-Secure world software (e.g., Linux) issues the SMC *PM_FPGA_READ*, or other affected functions (cf. AMD-SB-8008 [5]), all subsequent entities that handle and execute the service are Secure masters as Figure 3 shows. This means that they can access Secure memory that is not directly accessible to Non-Secure software. This has strong security implications, as the memory pointer passed as an argument of the SMC must be validated *in software* to ensure it does not break the hardware-based isolation provided by TrustZone. The Secure Monitor, in this case TF-A, is responsible for mediation and switching between the Secure and Non-Secure worlds on the APU, rejecting SMCs that may threaten TrustZone isolation. However, on the affected platforms, TF-A did not check the memory pointer passed as an argument of the SMC *PM_FPGA_READ*. In particular, memory pointers passed with the SMCs were not checked by TF-A to verify that Non-Secure software has access to the memory space. Thus, due to improper access control in TF-A, Non-Secure software was able to issue the SMC with a memory pointer that points to Secure memory and subsequently have this Secure memory region cleared. By leveraging this flaw, it enabled Non-Secure software (e.g., Linux) to clear control registers that require Secure access. This could subsequently be exploited to gain full read/write access to Secure memory, as explained in Section V-B.

When a *PM_FPGA_READ* is issued by Non-Secure software, it is handled by the Secure Monitor running in the Secure world at EL3. Then, the request is forwarded through an Inter-Processor Interrupt (IPI) to the Power Management Unit (PMU) without prior verification by TF-A of the address passed as an argument by Non-Secure software. The PMU is a hardware unit on the SoC responsible for system error handling, power management, and system initialization prior to boot. It provides other processors with runtime services such as cryptographic operations, eFUSE read and write operations, and PL management functions.

However, the PMU is not aware of the multiple security and execution states of the APU. The PMU, which receives the request sent by TF-A through an IPI, has no means to determine whether the request was initiated by Non-Secure or Secure software on the APU. In all cases, the IPI is sent by TF-A, which runs in Secure mode on the APU. Thus, it is the responsibility of the Secure Monitor, operating at the most

privileged level of the APU, to ensure consistency and security of requests from the Non-Secure world. In particular, it must guarantee that TrustZone's isolation will not be compromised before forwarding a request to the PMU or other processors, as the latter cannot enforce access control based on the security state of the software that initiated the request on the APU (e.g., Non-Secure software through a SMC).

It is worth mentioning that the PMU is a Secure master, and only Secure masters can access the PMU, preventing Non-Secure software from directly programming the PMU and bypassing the Secure Monitor. Upon receiving a *PM_FPGA_READ* request through an IPI, the PMU instructs the Configuration Security Unit (CSU) to read the PL configuration back to the designated memory location through the CSU DMA. The CSU is a specialized processor responsible for secure boot, secure key storage, cryptographic hardware acceleration, and PL configuration. It is important to mention that the CSU DMA generates only Secure accesses, granting it full access capabilities for both Secure and Non-Secure memory. Figure 3 shows the path of a *PM_FPGA_READ* SMC issued by a guest VM under Xen until the execution of the function by the CSU DMA.

In addition, TF-A (i.e., Secure Monitor) and PMU symbols were loaded into a hardware JTAG debugger and the execution flow from a SMC issued by Non-Secure software to the Secure CSU DMA executing the function was followed. It was found that the PMU also fails to ensure isolation at another level of granularity. On Zynq UltraScale+ SoC platforms, isolation can be implemented at a higher level of granularity through the XPPU and XMPU. These are hardware-based protection units that perform access control of memory transactions on a master-slave basis. They enable restricting masters (e.g., PMU, APU, DMA-capable devices) from accessing slaves (e.g., memory, IPIs, peripherals registers). The XPPU and XMPU can define access policies for different memory regions and peripheral registers, ensuring that only designated masters can access these slaves.

Upon receiving a *PM_FPGA_READ* request through an IPI, the PMU did not verify that the requesting processor (i.e., master) has access to the memory space (i.e., salve) pointed to by the memory pointer passed as an argument of the request (cf. CVE-2025-0038 [26] and AMD-SB-8008 [5]). This could be exploited by an untrusted processor to bypass the access control enforced by the XPPU and the XMPU. To that extent, the untrusted processor sends, through an IPI to the PMU, a *PM_FPGA_READ* request with a memory pointer, as an argument of the request, that points to a memory region that the untrusted processor is prevented from accessing by the XPPU and XMPU. In this way, a processor could leverage *PM_FPGA_READ* with a memory pointer targeting any memory region and subsequently have this memory region modified regardless of the access policies defined by the XPPU and XMPU. These protection units cannot prevent this since the master that initiates the request *PM_FPGA_READ* (e.g., non-secure APU) is different from the master that finally executes it (i.e., CSU DMA) and accesses the targeted memory region. For all the memory transactions involved in this request, the XPPU and XMPU cannot provide access control based on the master that initiated the request and the target slave accessed. Therefore, it is up to the PMU, which exposes services to different processors through IPIs, to ensure the consistency of requests before leveraging the CSU's services.

The lack of access control at different levels of granularity is listed below and also illustrated in Figure 3.

① As detailed in the previous section, there was improper access control for this SMC at the hypervisor level. Xen was forwarding any *PM_FPGA_READ* SMC regardless of the VM that issued the SMC and the pointer passed as a parameter (cf. zynqmp_eemi() in [28]). Thus, Xen failed to ensure VM isolation.

② Memory pointer passed with the SMC *PM_FPGA_READ* was not checked by TF-A (cf. *pm_fpga_read()* in [20]) to verify that the requesting Security state, particularly Non-Secure, has access to the targeted memory space (cf. CVE-2025-48507 [3]). Thus, TF-A failed to ensure TrustZone isolation.

③ The PMU did not verify that the requesting processor (i.e., master) has access to the memory region (i.e., slave) pointed to by the memory pointer passed as argument of the command *PM_FPGA_READ* (cf. CVE-2025-0038 [5]). Thus, XPPU and XMPU isolation could be bypassed through the services exposed by the PMU.

B. Exploitation of the Vulnerability to Gain Access to TrustZone's Secure World

This flaw could be leveraged by Non-Secure software to make cleared security control registers that require Secure access. To that extent, Non-Secure software sends a *PM_FPGA_READ* SMC with the address of the register to be cleared as an argument (*x1*). As the function is ultimately performed through a Secure DMA access by the CSU, the access succeeds and the Secure register is cleared. In this way, this flaw can be leveraged by Non-Secure software to clear *IOU_AXI_WPRTCN* and *IOU_AXI_RPRTCN* registers and subsequently gain full read/write access to Secure memory.

IOU_AXI_WPRTCN and *IOU_AXI_RPRTCN* require Secure access and enable the configuration of the AXI write protection type and read protection type respectively for IOP peripherals (e.g., SD Host Controller, Ethernet Controller, Quad-SPI Controller) [2]. In particular, these registers **enables configuring the security state (i.e., Secure, Non-Secure) of the master interfaces of these devices**. By default, all related devices are configured as Non-Secure masters through these registers. However, if these registers are cleared, all the peripherals' master interfaces are switched to the Secure state, meaning they can subsequently access Secure memory.

We were able to exploit the vulnerabilities in *PM_FPGA_READ* to clear these control registers and switch the SDHC master interface to Secure state. However, the slave interfaces (e.g., memory-mapped control registers) can still be accessed by Non-Secure software. Therefore, this could be exploited by Non-Secure software to carry out a DMA attack using the Secure SDHC to patch or exfiltrate Secure memory.

With the help of a hardware JTAG debugger, we verified that before exploiting the vulnerability, Non-Secure EL1 Linux and Non-Secure SDHC were not able to access the *PMU RAM* nor *IOU_AXI_WPRTCN* and *IOU_AXI_RPRTCN*, which are all Secure memory. Then, we verified that *IOU_AXI_WPRTCN* and *IOU_AXI_RPRTCN* can indeed be cleared through the SMC *PM_FPGA_READ*. Finally, we tested that the SDHC can subsequently patch the *PMU RAM*. A Proof of Concept (PoC) of the security vulnerability and the exploitation is available, and all artifacts for reproduction and testing are provided [23].

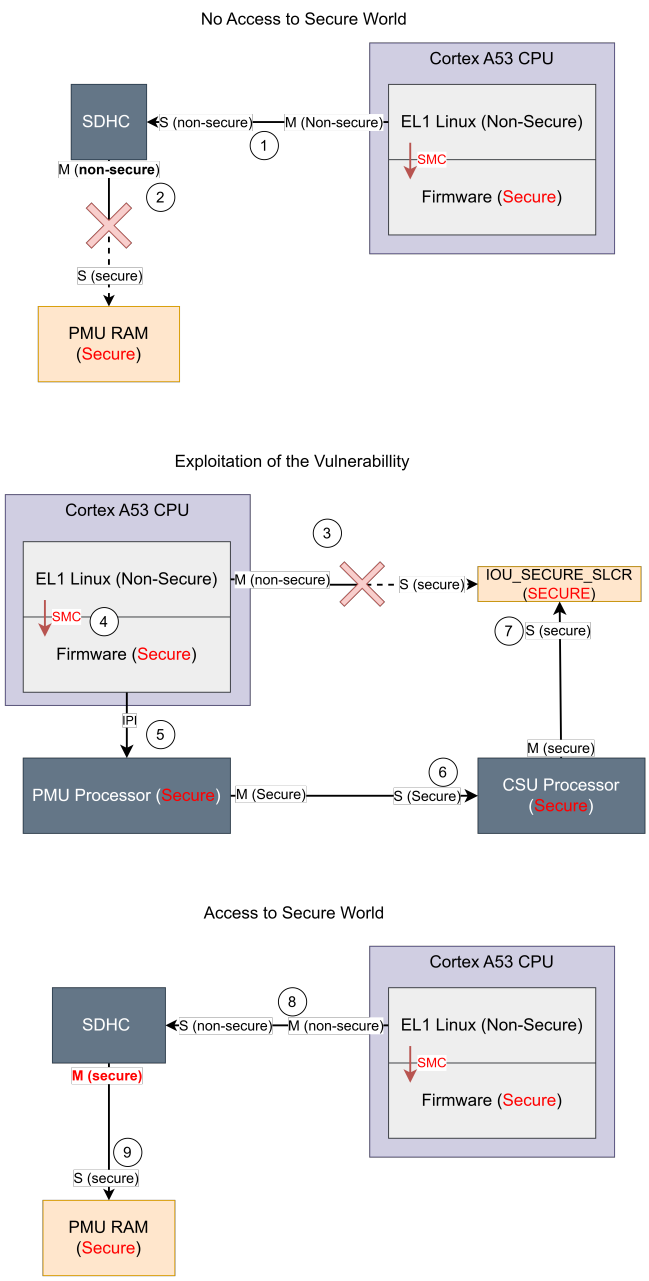

Fig. 4. Exploitation of the Vulnerability to Gain Full Access to Secure Memory from Non-Secure software

Below are the steps that we followed to exploit the security flaws and gain full access to the Secure world from Non-Secure software. Each step is also illustrated in Figure 4.

① Non-Secure EL1 software programs the SDHC through the Non-Secure slave interface (i.e., memory-mapped control registers) of the device.

② Instructed by Non-Secure EL1 software, Non-Secure SDHC tries to access *PMU RAM* but fails as this requires Secure access.

③ Non-Secure EL1 software tries to clear *IOU_AXI_WPRTCN*, which fails as the control register requires Secure access.

④ Non-Secure EL1 software issues the SMC *PM_FPGA_READ* to the Secure Monitor with *IOU_AXI_WPRTCN* as a SMC argument ($x1 >> 32$), indicating the target address from which the PL configuration must be read back.

⑤ The firmware forwards *PM_FPGA_READ* to the PMU through an IPI.

⑥ The PMU processor programs the CSU DMA to execute *PM_FPGA_READ* with the given target address (i.e., *IOU_AXI_WPRTCN*).

⑦ *IOU_AXI_WPRTCN* is cleared; write accesses generated by the SDHC are Secure.

⑧ Non-Secure software programs the SDHC through the Non-Secure slave interface (i.e., memory-mapped control registers) of the device.

⑨ As the SDHC master interface is Secure, it can access the *PMU RAM*, which is Secure. In this way, Non-Secure Linux can control the SDHC to exfiltrate or patch Secure memory.

1) Reducing the Attack Surface: Although disabled by default in the development tools or software, the XMPU and XPPU can be used to mitigate the exploitation detailed above and reduce the overall attack surface. Particularly, they can be used to limit the CSU DMA from accessing certain memory regions or to restrict untrusted processors from issuing IPIs to the PMU. However, they cannot be used to address the security vulnerabilities detailed at TF-A or PMU level. Indeed, the XMPU and XPPU implement access control based on the master that initiated a memory transaction and the targeted slave. However, they cannot provide access control based on the *initiator* of a request (e.g., Non-Secure APU), which is different from the master that finally executes the request (i.e., the CSU DMA). Specifically, the XPPU and XMPU cannot implement access control based on the processor that initiated PM_FPGA_READ and the memory pointer passed as an argument of the request, since there is no memory transaction corresponding to this master-slave association. The XPPU and XMPU could be used to prevent the APU from sending IPIs to the PMU or prevent the PMU from accessing the CSU. However, this would significantly impact system functionality, as the APU could not access any of the services provided by the PMU or the CSU, and the PMU could not access the services provided by the CSU.

VI. Conclusion

The SMC interface presents a large and modular API to the Non-Secure world, including to VMs. The support of the SMC interface in a virtualized system requires significant porting

efforts at the hypervisor level. The more this interface is supported, the more VMs can leverage features as they would in a native environment. However, this also significantly increases the attack surface for VMs. This interface is a particular attack vector as it exposes low-level functions to VMs that they could potentially leverage to break isolation. This is also because the firmware, which handles SMCs forwarded by the hypervisor, is not aware of VM isolation and has extended capabilities on the system. Moreover, the SMC interface exposes services specific to various entities (e.g., Trusted OS, external hardware units). Supporting them at the hypervisor level requires advanced knowledge of the specific APIs associated with each entity. At the time of writing, current hypervisors have limited support for the SMC interface. Nevertheless, analysis of the support for this interface at the hypervisor level enabled easy identification of security flaws that could be leveraged by VMs to break horizontal isolation.

The criticality of this interface is supported in the last section, which demonstrated that different entities failed to ensure isolation at various levels. Indeed, the SMC interface is particularly critical because functions are executed by distinct entities (i.e., hypervisor, firmware, hardware units) that must ensure isolation and access control at different levels of granularity. This is supported by the security vulnerabilities (e.g., CVE-2025-48507 [3], CVE-2025-0038 [26]) found in the firmware of Zynq UltraScale platforms, which enabled breaking TrustZone isolation and the access control provided by the XPPU and XMPU. The criticality of the SMC interface is emphasized as it was showed that Non-Secure software could exploit these security vulnerabilities to gain full read and write access to the Secure world. While existing hardware-based protection units on the platform can mitigate the attack surface, they cannot be used to address the security vulnerabilities found.

In conclusion, once the hypervisor forwards the SMC to the firmware, VM isolation is no longer guaranteed, as the firmware is unaware of the virtualized systems in the Non-Secure world. Similarly at a higher level of granularity, once the Secure Monitor on the APU forwards a request to another Secure unit on the platform, TrustZone isolation can no longer be guaranteed, since these units are not aware of the different security states of the APU. This has strong security implications, as the SMC from Non-Secure software must be validated **in software** to ensure the request does not break the hardware-based isolation provided by TrustZone or protection units.

References

[1] AMD. *XilFPGA Library v6.7 - Standalone Library Documentation: BSP and Libraries Document Collection*, November 2024. UG643.

[2] AMD. *Zynq UltraScale+ Devices Register Reference*, March 2024. UG1087 Rev 1.1.

[3] AMD. CVE-2025-48507, 11 2025.

[4] AMD. *Zynq UltraScale+ Device Technical Reference Manual*, March 2025. UG1085 Rev2.5.

[5] AMD. Zynq UltraScale+ SoC Overwriting Protected Memory Regions Through PMU Firmware. https://www.amd.com/en/resources/product-security/bulletin/amd-sb-8008.html, July 2025. AMD-SB-8008.

[6] Arm Limited. *Arm Management Mode (MM) Interface Specification*, December 2016. Arm DEN 0060 Ver. A.

[7] Arm Limited. *Arm PCI Configuration Space Access Firmware Interface*, December 2020. Arm DEN 0115 Ver. 1.0.

[8] Arm Limited. *Arm Errata Management Firmware Interface*, October 2022. Arm DEN 0100 Ver. 1.

[9] Arm Limited. *Arm Paravirtualized Time for Arm-based Systems*, June 2022. Arm DEN 0057 Ver. A.b.

[10] Arm Limited. *Arm True Random Number Generator (TRNG) Firmware Interface*, January 2022. Arm DEN 0098 Ver. 1.

[11] Arm Limited. *Software Delegated Exception Interface (SDEI)*, January 2023. Arm DEN 0054 Ver. C.

[12] Arm Limited. *Arm Firmware Framework for Arm A-profile (FF-A)*, November 2024. Arm DEN 0077 Ver. 1.3.

[13] Arm Limited. *Arm Power State Coordination Interface (PSCI)*, October 2024. Arm DEN 0022 Ver. F.b.

[14] ARM Limited. *ARM SMC Calling Convention (SMCCC)*, October 2024. ARM DEN 0028 Ver. 1.6G.

[15] Arm Limited. *DRTM Architecture for Arm*, October 2024. Arm DEN 0113 Ver. 1.1.

[16] Jean De Bonfils Lavernelle, Pierre-François Bonnefoi, Benoît Gonzalvo, and Damien Sauveron. DMA: A Persistent Threat to Embedded Systems Isolation. In *2024 IEEE 23rd International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*, pages 101–108, 2024.

[17] John Ernberg. xen/arm: Add imx8q{m,x} platform glue. https://github.com/xen-project/xen/commit/71d4356544ee49eeb6b58383f15eef330c7e8452, April 2024.

[18] Peng Fan. only allow root cell to use sip. https://github.com/nxp-imx/imx-jailhouse/commit/e4a40952591db377ceb9965b85ee0c4430245801, November 2024.

[19] Peng Fan. only allow root cell to use Trusted SMC. https://github.com/nxp-imx/imx-jailhouse/commit/a41801cafd26797250e25e56272c111ab28849fb, December 2024.

[20] Arm Trusted Firmware. GitHub Trusted Firmware-A - Zynq Power Management API. https://github.com/ARM-software/arm-trusted-firmware/blob/master/plat/xilinx/zynqmp/pm_service/zynqmp_pm_api_sys.c, 2025. (Accessed on 10/02/2025).

[21] Bao Hypervisor. Bao - a lightweight static partitioning hypervisor. https://github.com/bao-project/bao-hypervisor, 2025. (Accessed on 10/11/2024).

[22] NXP i.MX. i.MX Jaihouse Hypervisor. https://github.com/nxp-imx/imx-jailhouse, 2025.

[23] Jean de Bonfils Lavernelle. Proof of Concept for CVE-2025-0038 (Zynq UltraScale+ SoC). https://github.com/jdbonfils/CVE-2025-0038-PoC, July 2025.

[24] Arm Limited. Trusted Firmware-A (TF-A) Documentation — 4.20. RMM-EL3 Communication interface. https://trustedfirmware-a.readthedocs.io/en/stable/components/rmm-el3-comms-spec.html. (Accessed on 12/25/2024).

[25] Arm Limited. Trusted Firmware-A (TF-A) Documentation — 4.22. Vendor Specific EL3 Monitor Service Calls. https://trustedfirmware-a.readthedocs.io/en/stable/components/ven-el3-service.html. (Accessed on 12/25/2024).

[26] MITRE Corporation. CVE-2025-0038. TOBEAPPENDED, July 2025.

[27] Linux Kernel Organization. Generic vm interface. https://www.kernel.org/doc/Documentation/virt/kvm/devices/vm.rst. (Accessed on 12/03/2025).

[28] Stefano Stabellini. xen: restrict PM_FPGA_READ to Dom0. https://github.com/Xilinx/xen/commit/c9de96c0cbe9b2f2aa5e55a0e1e645ca72865102, October 2024.

[29] Xilinx. Xilinx Xen. https://github.com/Xilinx/xen/tree/xlnx_rebase_4.18, 2025.